

Dockers

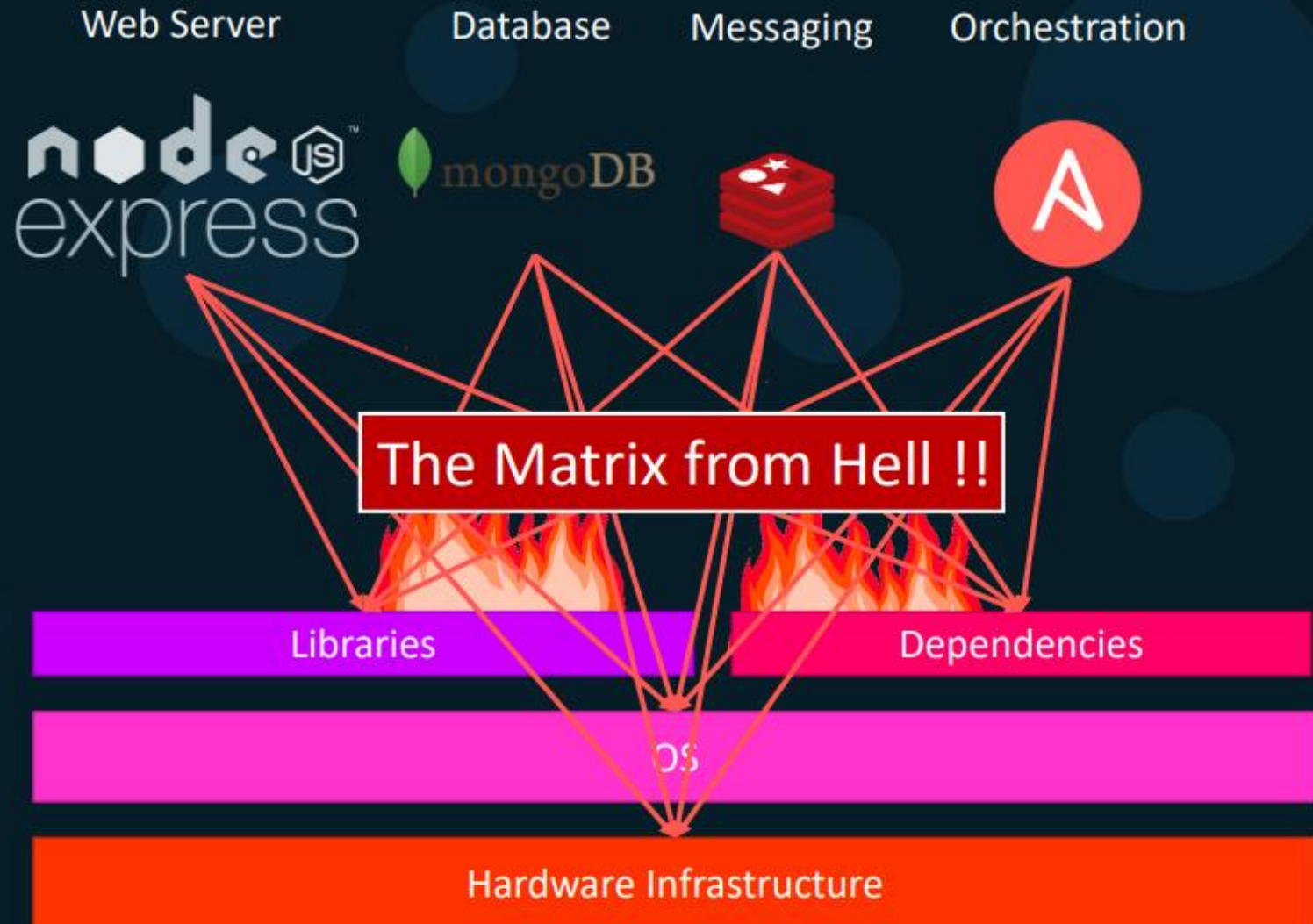
Overview

Why Docker

- “it works on my machine”
- “Complex Setup”
- “Dependency conflicts”
 1. You have multiple projects, each needing different versions of the same library or even different languages.
 2. Virtual environments (like venv) help, but they only isolate Python packages, not OS-level dependencies (like system libraries or databases).

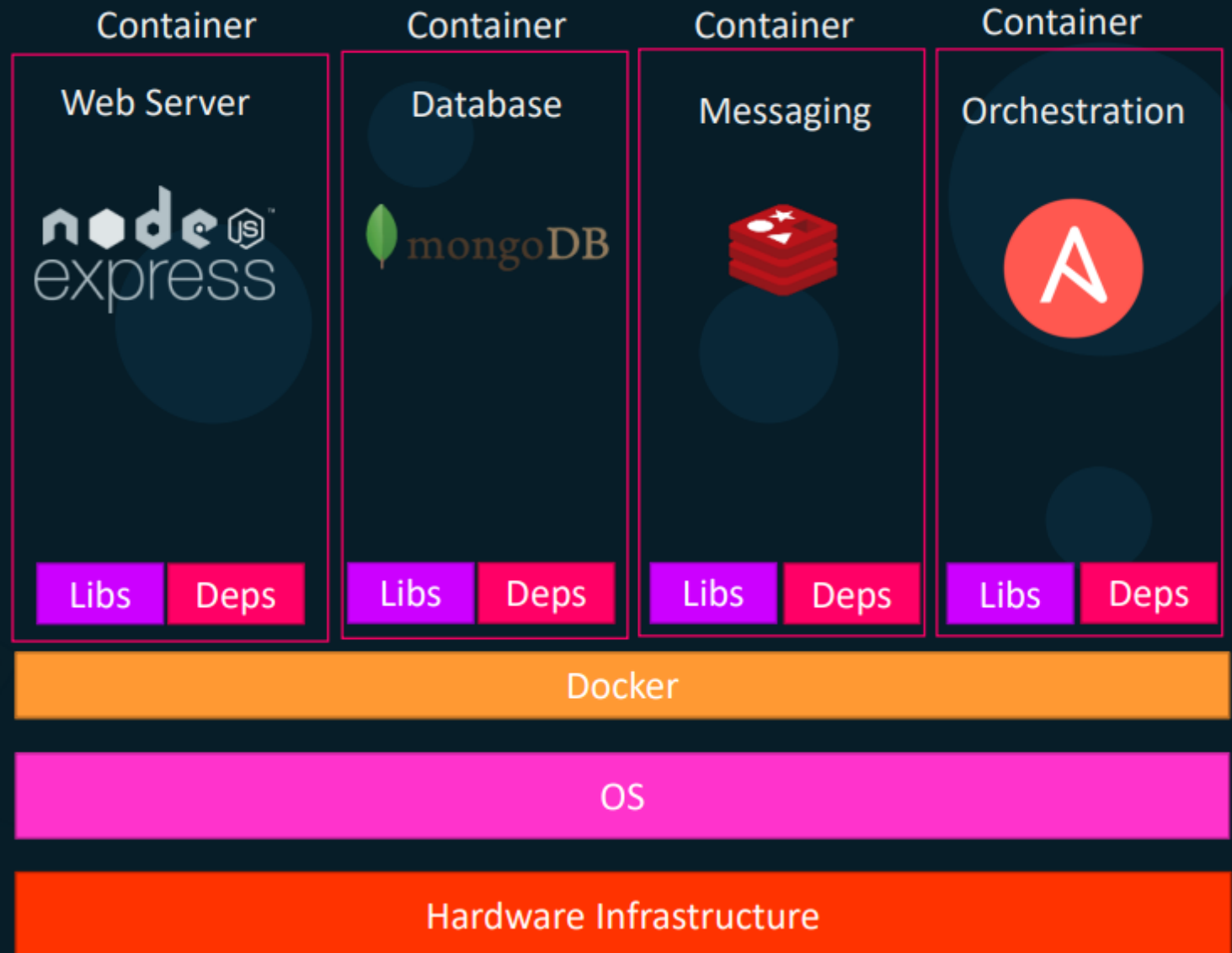
Why do you need docker?

- Compatibility/Dependency
- Long setup time
- Different Dev/Test/Prod environments



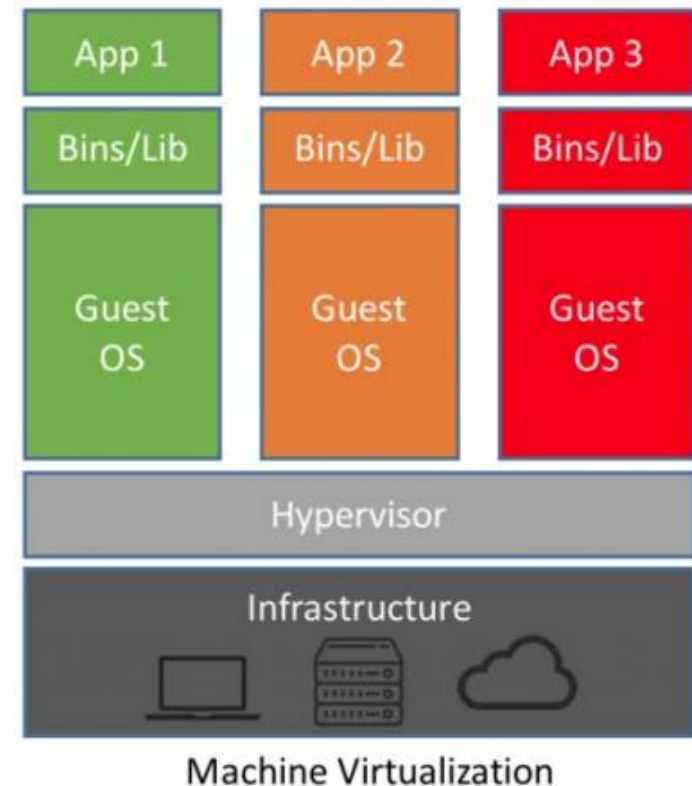
What can it do?

- Containerize Applications
- Run each service with its own dependencies in separate containers



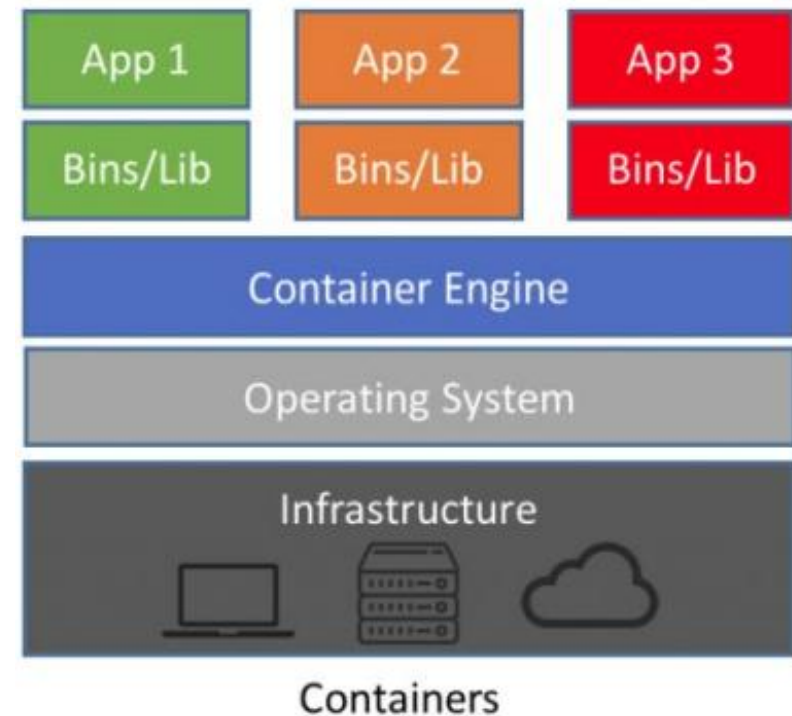
Virtual Machines Vs Containers

- VMs run applications inside a guest Operating System, which runs on virtual hardware powered by the host server's OS.
- VMs are great at providing full process isolation for applications BUT the computational overhead spent virtualizing hardware for a guest OS to use is substantial.



Virtual Machines Vs Containers

- By leveraging the low-level mechanics of the host operating system, containers provide most of the isolation of virtual machines at a fraction of the computing power.
- Docker is a container-based system for your applications



Dockers

- Docker is a tool that allows developers, sysadmins etc. to easily deploy their applications in a sandbox (called containers) to run on the host operating system.
- Docker allows users to package an application with all of its dependencies into a standardized unit for software development.

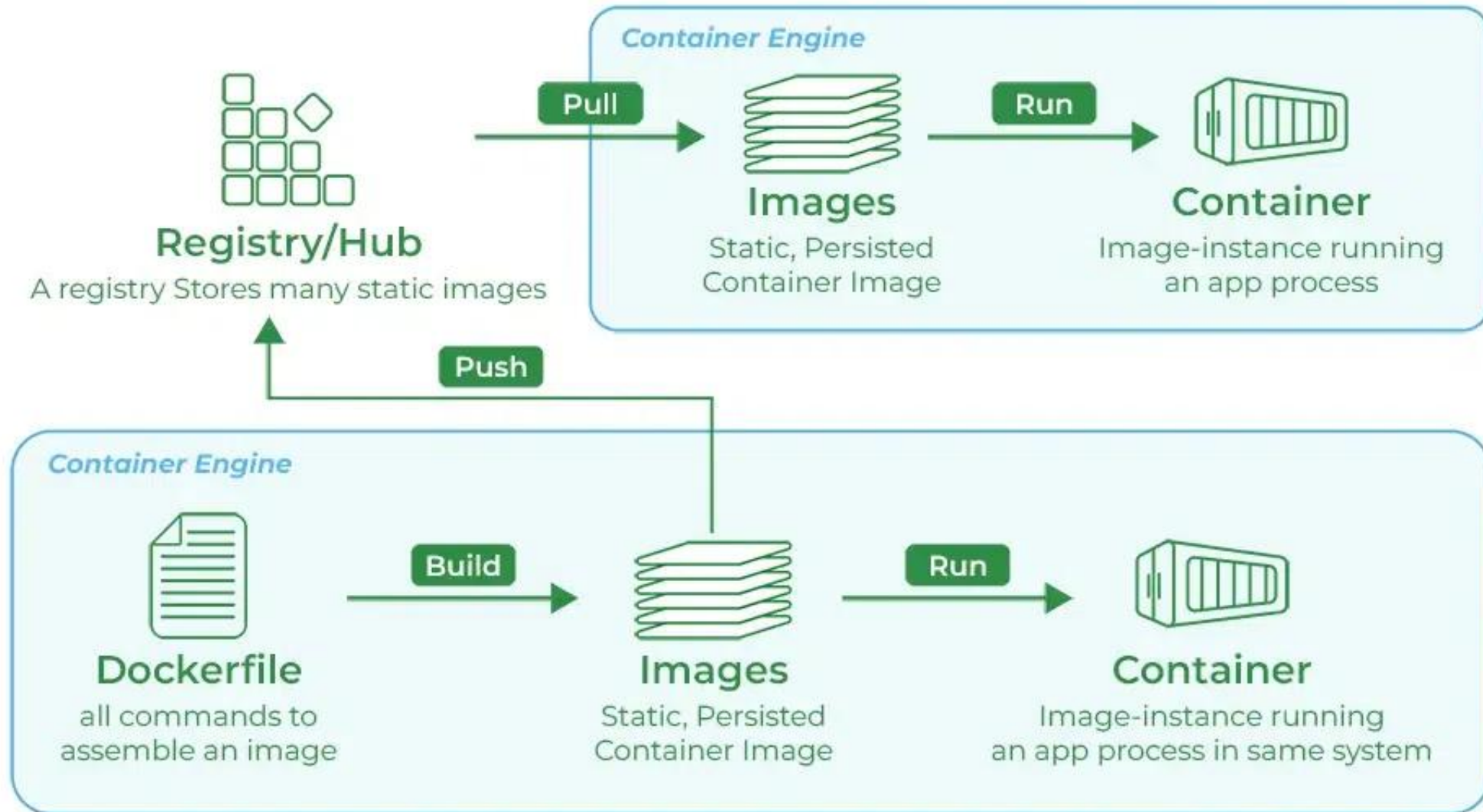
How Docker works?

- Docker solves: “How can I make sure my app runs *exactly the same way* on every machine, without manual setup or dependency issues?”

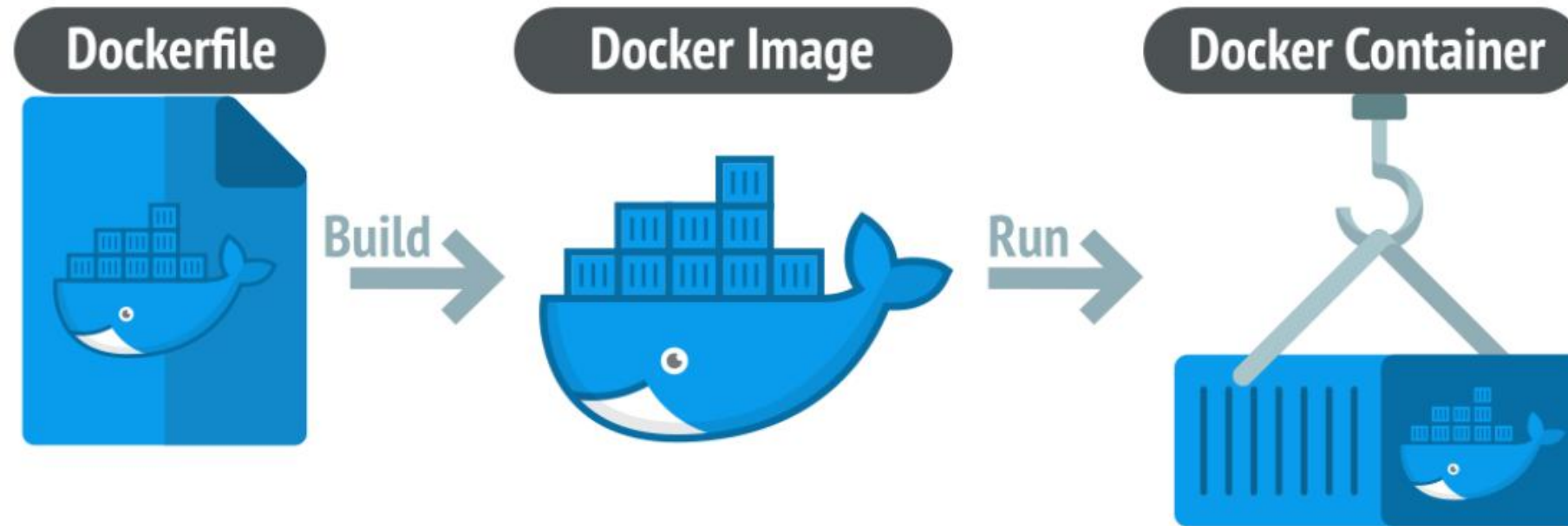
By using:

- **Images** → Packaged app + dependencies
- **Containers** → Isolated running environments
- **Docker Hub** → Share and distribute images
- **Compose / Swarm / Kubernetes** → Orchestrate multiple containers

How Docker works?



How Docker works?



Build and Deploy First Container on DockerHub

Docker Installation

- Docker Desktop: <https://www.docker.com/products/docker-desktop>
- Two options to install: ***hyper-v*** or ***wsl***
- Virtualization must be enabled for hyper-v installation.

Code: Hello Flask App

```
from flask import Flask
```

```
app=Flask(__name__)
```

```
@app.route("/",methods=["GET"])
```

```
def hello():
```

```
    return "<h1>Hello from Flask!</h1>"
```

```
if __name__ == "__main__":
```

```
    # Only for local debugging; container uses gunicorn.
```

```
    app.run(host="0.0.0.0", port=5000)
```

Code: Requirements file

Requirements file contains:

flask

Dockerfile

Small python base image

From python:3.11-slim

Copy current directory contents to leverage Docker cache

COPY . /app

Set working directory

WORKDIR /app

Install dependencies

RUN pip install -r requirements

Command to run app.py file

CMD python app.py

Important Commands

- `docker -v` Check docker version
- `docker pull docker/getting-started` pull container from dockerHub
- `docker images` Check docker images
- `docker ps` Running images
- `docker stop [container_id]` Stop container by id
- `docker build -t [image_name] .` Build Image from directory
- `docker run -p 5000:5000 welcome-app` Run a container
- `docker run --rm -p 5000:500 welcomeapp` Run a container 2
- `docker logs container_id` View logs of container by id

Important Commands

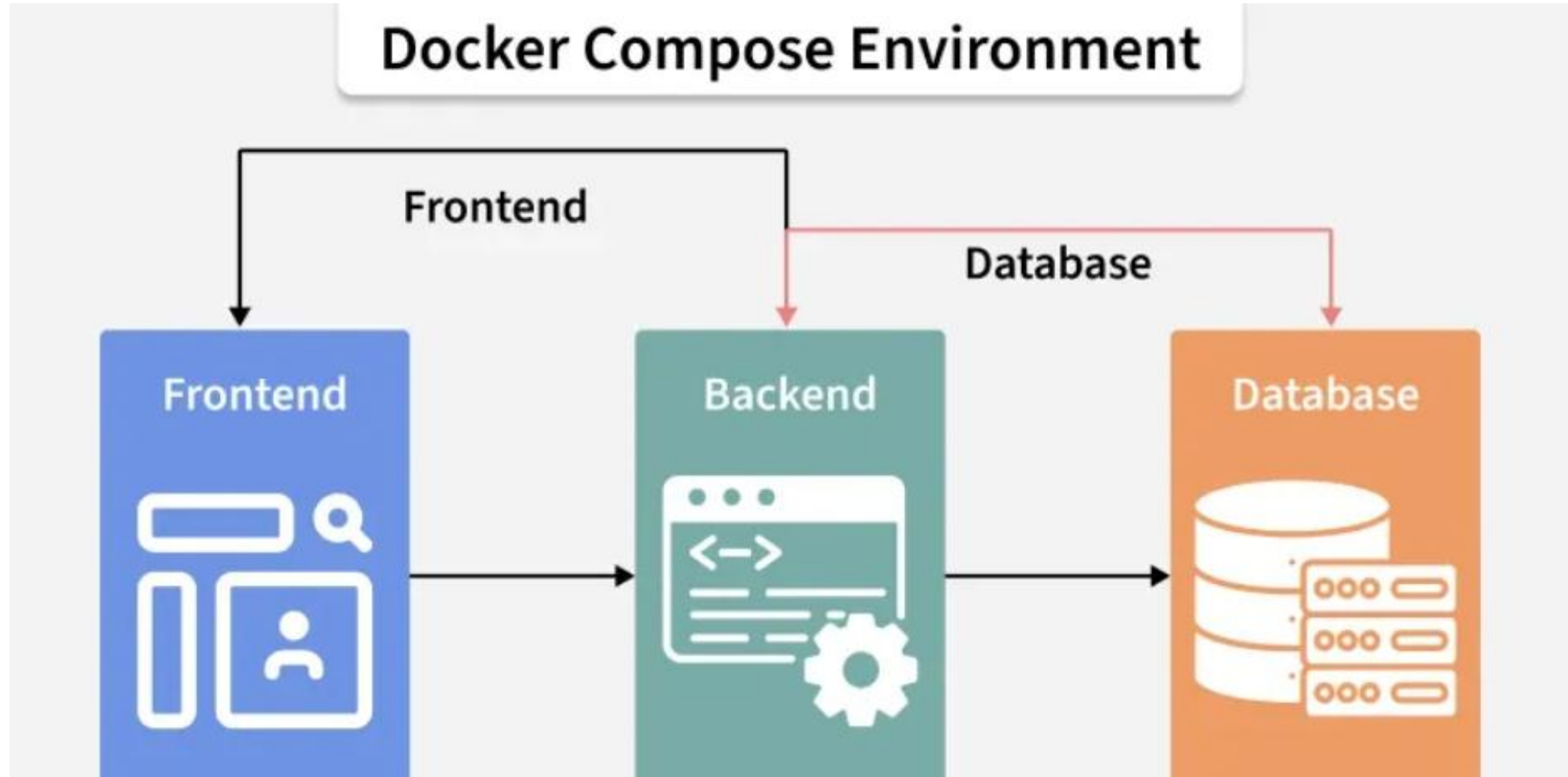
- `docker login` Login to dockerHub
- `docker image rm -f [image_name]` Removes an image
- `docker build -t [repository_name]` . built a docker image
- `docker push [repository_name]` push container to
dockerHub

Docker Compose

Docker Compose

- **Docker Compose** is an orchestration tool for defining and running multi-container Docker applications.
- Imagine you have a complex application with separate components—**a web server, a backend API, a database, and a caching layer**. Each component runs in its own isolated container. The challenge isn't just running these containers individually, but making them work together as a single, cohesive application.

Docker Compose



Challenges with Multi-Container Setup

- **Complex Configuration:** Running each container requires long and complex docker run commands with flags for **networking**, **ports**, **volumes**, and **environment variables**. This becomes difficult to manage and reproduce consistently.
- **Manual Networking:** To make containers talk to each other, you would have to manually create Docker networks, find the internal IP address of each container.

Docker Compose Services

- In Docker Compose, every component of your application operates as a separate service, with each service running a single container tailored to a specific role—such as a database, web server, or cache
- These services are defined within the `services` section of the *`docker-compose.yml`* file.

Key Service Configuration Options

- **Image:** Image option defines which Docker image we are going to use by the service from the Docker Hub or any other registry.
- **Build:** Instead of pulling an image, you can build one locally by specifying a directory containing a Dockerfile. The build is ideal for including custom code in your application.
- **Ports:** This setting maps a container's internal ports to those on the host machine, enabling access to services from outside the container.

Key Service Configuration Options

- **Volumes:** Volumes attach persistent storage to a service ensuring that the data remains accessible even when a container restart.
- **Environment:** Environment variables allow you to pass configurations or sensitive information, like database credentials or API keys, to the service.
- **Depends_on:** Depends_on controls the startup order of services, ensuring that certain containers are running before others begin.

docker-compose.yml

Example 1

```
version: '3.8'           # latest version of Docker compose
services:                 # Each service will work as container
  db:                     # Container 1
    image: postgres:13    # Base Image
    environment:          # Authentication Flags to DB
      - POSTGRES_USER=user
      - POSTGRES_PASSWORD=pass
  volumes:                # For Persistent Storage
    - db_data:/var/lib/postgresql/data
```

docker-compose.yml

Example 1

version: '3.8'

services:

db:

web: # Container 2

build: ./web # Use /web to build image

ports:

- "5000:5000" # Maps the Flask app's internal port 5000 to your host port 5000

volumes:

- web_data:/usr/src/app # Persistent Storage

environment:

- DATABASE_URL=postgres://user:pass@db:5432/mydb # Database Authentication

depends_on:

- db # RUN db before web

volumes:

db_data:

web_data:

docker-compose.yml

Example 2

version: '3.8'

services:

db:

image: postgres:13

networks:

- backend

web:

image: nginx:latest

networks:

- frontend

- backend

ports:

- "80:80"

.....

docker-compose.yml

Example 2

```
version: '3.8'
```

```
services:
```

```
  db:
```

```
    ....
```

```
  web:
```

```
    ....
```

```
networks:
```

```
  frontend:
```

```
    driver: bridge
```

```
  backend:
```

```
    driver: bridge
```

```
    ipam:
```

```
      config:
```

```
        - subnet: 172.16.238.0/24
```